

ANALISIS Y METODOLOGIA DE SISTEMAS

Trabajo Parcial 1

StockPar

Sistema de gestion de repuestos importados con precios en dolares

Dato	Informacion
Alumno	Nahuel Ignacio Ezquerra
Comision	ACN4AP
Materia	Analisis y Metodologia de Sistemas
Trabajo	Trabajo Parcial 1
Sistema	StockPar
Año	2026

Indice

1. Resumen ejecutivo
2. Descripcion del problema
3. Propuesta de valor
4. Benchmarking
5. Alcance y modelo de funcionamiento
6. Historias de usuario
7. Planificacion Kanban
8. Diagrama de contexto Nivel 0
9. DFD Nivel 1
10. Diagrama Entidad Relacion
11. Trazabilidad entre DFD y DER
12. Modelo de monetizacion
13. Diagrama de clases UML
14. Diseño de API REST
15. Prototipo de codigo POO
16. Repositorio GitHub
17. Pruebas y criterios de calidad
18. Conclusiones
19. Bibliografia
20. Anexos

1 Resumen ejecutivo

StockPar es una propuesta de sistema para una distribuidora argentina de repuestos importados. Centraliza el catalogo, los proveedores, el stock, la cotizacion del dolar y los presupuestos comerciales. El sistema resuelve el riesgo de utilizar planillas desactualizadas y de calcular manualmente precios de venta en pesos a partir de costos expresados en dolares.

La solucion contempla dos perfiles principales: Administrador y Vendedor. El Administrador mantiene los productos, proveedores, niveles de stock y cotizaciones del dolar. El Vendedor consulta disponibilidad, calcula precios y genera cotizaciones cuyo valor queda historizado. El diseño incluye modelos de procesos, datos, clases, API REST y un prototipo orientado a objetos.

2 Descripcion del problema

La empresa administra compras en USD y ventas en ARS mediante planillas y registros manuales. La variacion cambiaria vuelve rapidamente obsoletos los precios y obliga a los vendedores a repetir calculos, consultar archivos diferentes o reutilizar valores anteriores. Al mismo tiempo, no existe una unica fuente confiable para conocer el stock real.

- Cotizaciones con precios incorrectos o vencidos.
- Perdidas por vender por debajo del costo actualizado.
- Demoras en la atencion al cliente.
- Diferencias entre el stock fisico y el registrado.
- Dependencia de archivos personales y falta de trazabilidad.
- Dificultad para detectar productos que requieren reposicion.

2.1 Objetivo general

Diseñar un sistema centralizado que permita gestionar repuestos importados, calcular precios de venta en pesos con una cotizacion del dolar identificable, controlar el stock y conservar el historial de cotizaciones realizadas.

2.2 Objetivos especificos

- Registrar repuestos y proveedores con datos consistentes.
- Mantener un historial de cotizaciones del dolar.
- Calcular precios de venta aplicando costo, tipo de cambio y margen.
- Consultar stock actual y alertas de stock minimo.
- Generar cotizaciones con items, cantidades, precios y total.
- Conservar quien genero cada operacion y con que valor del dolar.

3 Propuesta de valor

StockPar ofrece una fuente unica de informacion para vender repuestos importados con rapidez y menor riesgo. La principal diferencia es que vincula cada presupuesto con la

cotizacion del dolar utilizada, evitando que un cambio posterior modifique el historial comercial.

Beneficio	Resultado esperado
Precios trazables	Cada cotizacion conserva el valor del dolar y los precios aplicados.
Menos errores	El sistema automatiza conversion, margen, subtotal y total.
Control operativo	El stock y el minimo se consultan desde un unico lugar.
Rapidez comercial	El vendedor encuentra productos y genera presupuestos en menos pasos.
Informacion historica	Se puede revisar quien cotizo, cuando y bajo que condiciones.

3.1 Formula de precio

Precio base en ARS = precio de compra en USD x cotizacion del dolar.

Precio de venta en ARS = precio base en ARS x (1 + margen de ganancia / 100).

El resultado debe redondearse segun la politica comercial. El precio unitario utilizado se copia al detalle de la cotizacion para que no cambie cuando se actualicen el dolar o el margen.

4 Benchmarking

Se analizaron Boxer Gestion y Tezzi por su orientacion a casas de repuestos y gestion comercial. El objetivo es identificar funciones habituales y precisar el diferencial de StockPar.

Caracteristica	Boxer Gestion	Tezzi	StockPar
Stock	Si	Si	Si
Repuestos y proveedores	Si	Si	Si
Presupuestos	Si	Si	Si
Importacion de listas	No destacada	Si	Futura
Facturacion electronica	Si	Si	Fuera del MVP
Costo principal en USD	No destacado	No destacado	Si
Conversion USD a ARS	No destacada	No destacada	Si
Dolar historico por cotizacion	No identificado	No identificado	Si

Conclusion: StockPar no busca competir inicialmente con un ERP completo. Se enfoca en resolver de forma simple y trazable el precio de repuestos importados y el control de stock. Facturacion electronica e importacion masiva quedan como evolucion posterior.

5 Alcance y modelo de funcionamiento

5.1 Alcance del producto minimo viable

- Autenticacion y roles Administrador/Vendedor.
- ABM de proveedores y repuestos.
- Registro e historial de cotizaciones del dolar.
- Calculo de precio vigente en ARS.
- Consulta por codigo, nombre o marca.
- Gestion de stock y reporte de stock bajo.
- Creacion, consulta y cambio de estado de cotizaciones.
- Descuento de stock solo cuando la cotizacion se confirma como venta.

5.2 Fuera del alcance inicial

- Facturacion fiscal electronica.
- Contabilidad y cuentas corrientes.
- Compras automaticas a proveedores.
- Integracion bancaria.
- Comercio electronico publico.
- Aplicacion movil nativa.

5.3 Reglas de negocio

RN	Regla
RN01	El codigo de repuesto y el email de usuario deben ser unicos.
RN02	Solo el Administrador puede modificar repuestos, proveedores y cotizacion del dolar.
RN03	No se permiten costos, margenes, cantidades ni stock negativos.
RN04	Una cotizacion debe tener al menos un detalle.
RN05	Cada cotizacion registra el usuario y el valor del dolar utilizado.
RN06	El precio unitario y subtotal se guardan como valores historicos.
RN07	Confirmar una venta descuenta stock dentro de una transaccion.
RN08	No puede confirmarse una venta sin stock suficiente.
RN09	Un producto con stock actual menor o igual al minimo se informa como bajo stock.
RN10	Los registros referenciados se desactivan en lugar de eliminarse fisicamente.

5.4 Flujo principal

1. El Administrador registra proveedores y repuestos.

2. El Administrador carga una nueva cotizacion del dolar.
3. El Vendedor busca productos y consulta precio y disponibilidad.
4. El Vendedor agrega cantidades y datos del cliente.
5. StockPar calcula subtotales y total, y guarda el presupuesto.
6. Si el cliente acepta, el Vendedor confirma la venta.
7. StockPar valida y descuenta el stock, dejando trazabilidad.

6 Historias de usuario

ID	Como	Quiero	Para	Criterios de aceptacion
HU01	Administrador	registrar y actualizar repuestos	mantener el catalogo correcto	Codigo unico; datos obligatorios; costo y stock no negativos; consulta posterior.
HU02	Administrador	registrar la cotizacion del dolar	calcular precios con un valor vigente	Valor mayor que cero; fecha y fuente; historial inalterable.
HU03	Vendedor	buscar repuestos y consultar precio y stock	responder al cliente rapidamente	Busqueda por codigo, nombre o marca; precio calculado; aviso si no existe.
HU04	Vendedor	generar una cotizacion con varios repuestos	entregar un presupuesto trazable	Uno o mas detalles; subtotales y total; usuario, fecha y dolar registrados.
HU05	Administrador	definir y consultar stock minimo	anticipar reposiciones	Comparacion automatica; listado de productos bajo minimo.

6.1 Priorizacion MoSCoW

Prioridad	Funciones
Debe tener	Login, repuestos, proveedores, dolar, stock, cotizaciones.
Deberia tener	Alertas de stock, exportacion PDF, auditoria basica.
Podria tener	Importacion de listas, notificaciones y panel de indicadores.
No por ahora	Facturacion fiscal, contabilidad y tienda en linea.

7 Planificacion Kanban

El tablero usa las columnas Pendiente, En proceso, Revision y Terminado. Cada tarjeta incluye responsable, historia relacionada, criterios de aceptacion y evidencia.

Estado	Tarjetas sugeridas
Terminado	Problema, propuesta de valor, benchmarking, alcance e historias.
Revision	Contexto, DFD, DER, trazabilidad y UML.
En proceso	API REST, codigo POO y pruebas.
Pendiente	Repositorio publico, carga de evidencias y presentacion final.

7.1 Definicion de terminado

- El entregable responde a una historia o requisito.
- Los nombres coinciden en DFD, DER, UML y API.
- Las reglas principales estan validadas.
- Existe una evidencia revisable: diagrama, documento, codigo o prueba.
- La tarjeta fue revisada antes de pasar a Terminado.

8 Diagrama de contexto Nivel 0

El Nivel 0 representa StockPar como un unico proceso y muestra sus intercambios con Administrador, Vendedor y Cliente. El Cliente puede comunicarse a traves del Vendedor; se conserva como entidad para mostrar el origen del pedido y el destino del presupuesto.

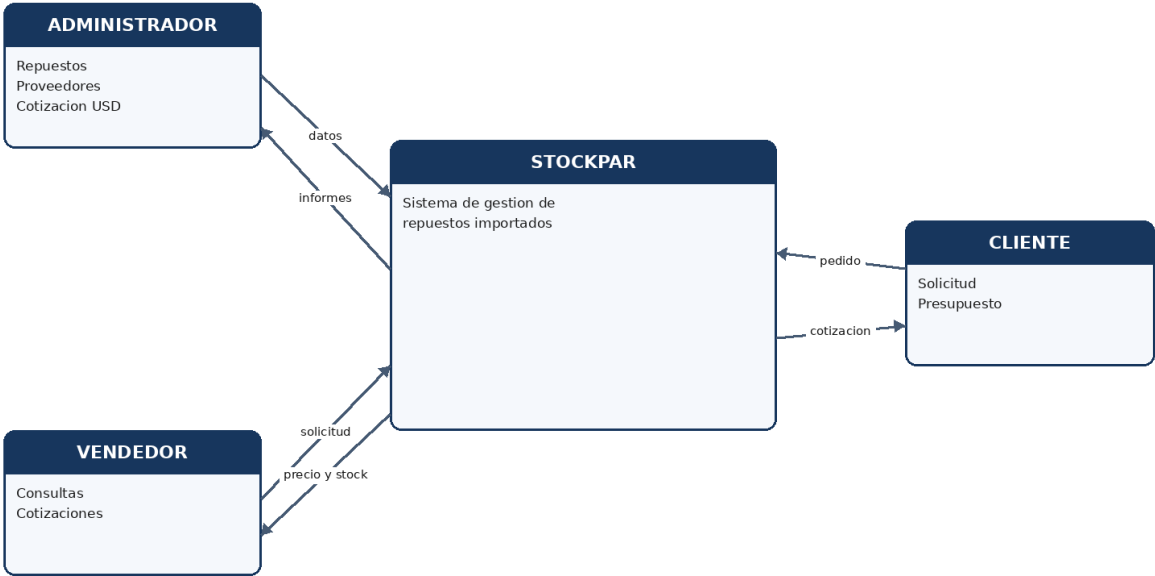


Figura 1. Diagrama de contexto de StockPar

Entidad	Envía al sistema	Recibe del sistema
Administrador	Repuestos, proveedores, dolar y ajustes de stock	Informes, precios y alertas
Vendedor	Consultas, datos del cliente, productos y cantidades	Precio, stock y cotizacion
Cliente	Solicitud y datos de contacto	Presupuesto generado

9 Diagrama de flujo de datos Nivel 1

El DFD descompone el sistema en cuatro procesos. Se agrega el almacen de Usuarios de forma logica en la trazabilidad, aunque el diagrama principal prioriza los almacenes del dominio comercial.

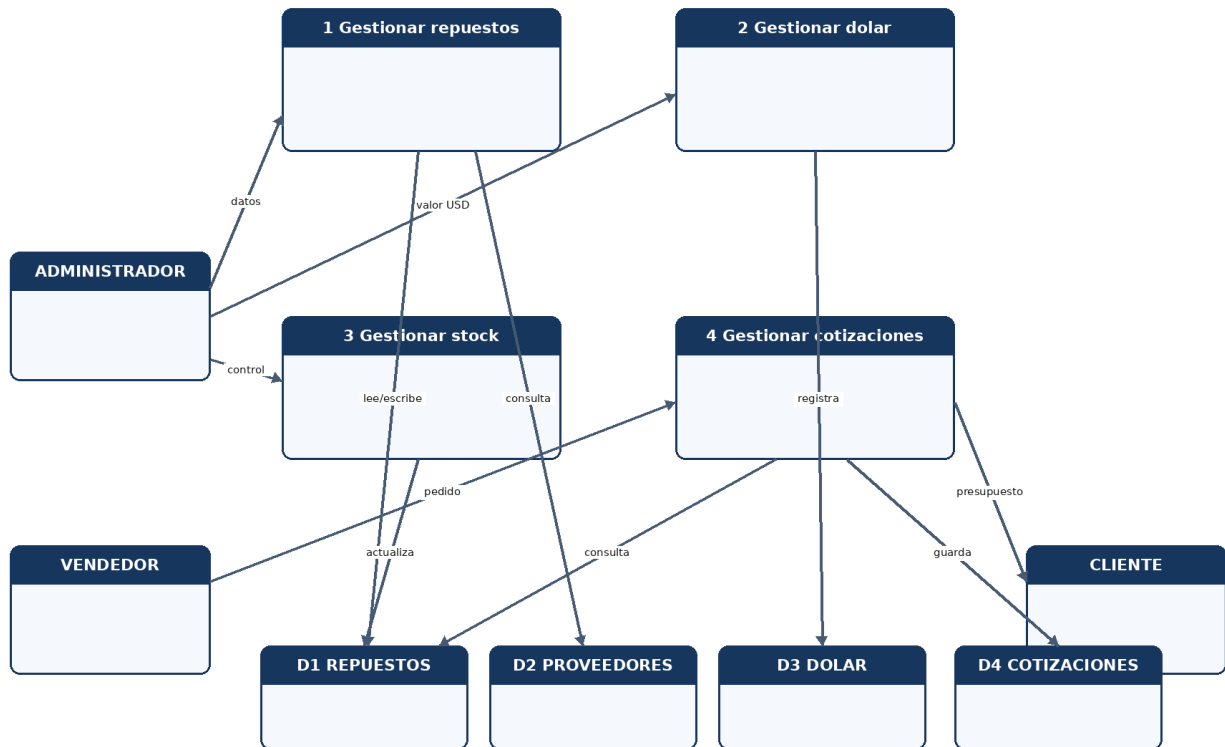


Figura 2. DFD Nivel 1 de StockPar

Proceso	Entradas	Almacenes	Salidas
1 Gestionar repuestos	Altas, cambios y consultas	D1 Repuestos, D2 Proveedores	Catalogo actualizado
2 Gestionar dolar	Nuevo valor y fuente	D3 Cotizaciones del dolar	Cotizacion vigente
3 Gestionar stock	Ajustes y confirmaciones	D1 Repuestos	Disponibilidad y alertas
4 Gestionar cotizaciones	Cliente, items y cantidades	D1 Repuestos, D3 Dolar, D4 Cotizaciones	Presupuesto e historial

9.1 Balance con el Nivel 0

Todos los flujos externos del Nivel 0 aparecen desarrollados en el Nivel 1: el Administrador mantiene datos y recibe alertas; el Vendedor consulta y cotiza; el Cliente origina una solicitud y recibe un presupuesto. No se incorporan entidades externas nuevas.

10 Diagrama Entidad Relacion

El DER define seis entidades principales. DETALLE_COTIZACION resuelve la relacion muchos a muchos entre COTIZACION y REPUESTO. COTIZACION incorpora id_cotizacion_dolar para conservar el valor utilizado.

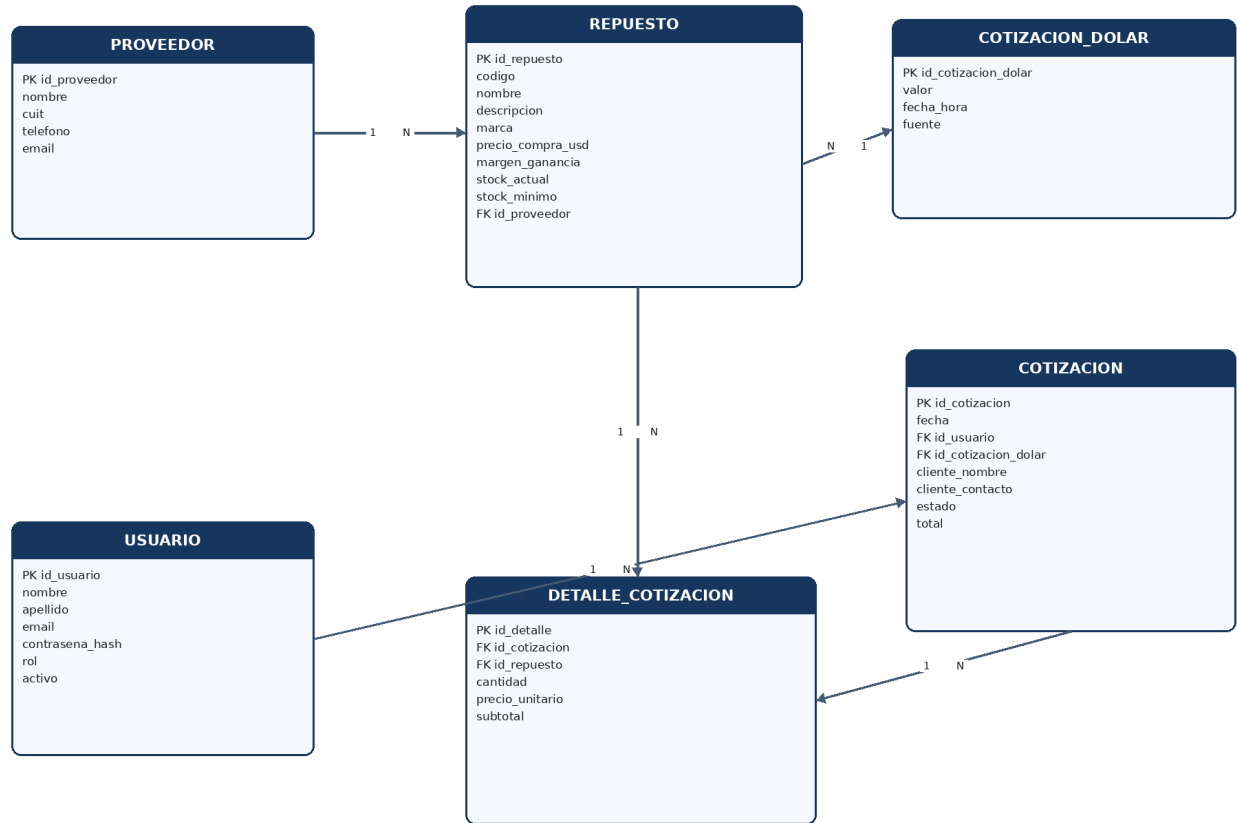


Figura 3. Diagrama Entidad Relacion de StockPar

10.1 Diccionario resumido

Entidad	Clave primaria	Claves foraneas	Proposito
USUARIO	id_usuario	-	Identidad, rol y estado del usuario.
PROVEEDOR	id_proveedor	-	Origen comercial de los repuestos.
REPUESTO	id_repuesto	id_proveedor	Catalogo, costo, margen y stock.
COTIZACION_DOLAR	id_cotizacion_dolar	-	Valor historico del tipo de cambio.
COTIZACION	id_cotizacion	id_usuario, id_cotizacion_dolar	Cabecera y datos del cliente.
DETALLE_COTIZACION	id_detalle	id_cotizacion, id_repuesto	Cantidad, precio historico y subtotal.

10.2 Cardinalidades

- PROVEEDOR 1 a N REPUESTO.
- USUARIO 1 a N COTIZACION.
- COTIZACION_DOLAR 1 a N COTIZACION.
- COTIZACION 1 a N DETALLE_COTIZACION.
- REPUESTO 1 a N DETALLE_COTIZACION.

10.3 Restricciones recomendadas

- UNIQUE en usuario.email, proveedor.cuit, repuesto.codigo.
- CHECK para valores monetarios, stock y cantidad no negativos.
- Índice sobre repuesto.nombre, repuesto.marca y cotizacion.fecha.
- DECIMAL para importes; nunca FLOAT.
- Hash de contraseña; nunca guardar la contraseña en texto plano.

11 Trazabilidad entre DFD y DER

La trazabilidad demuestra que los datos consumidos o producidos por los procesos tienen una estructura persistente en el DER.

Proceso DFD	Datos que utiliza	Entidades DER	Historias
1 Gestionar repuestos	Producto y proveedor	REPUESTO, PROVEEDOR	HU01
2 Gestionar dolar	Valor, fecha y fuente	COTIZACION_DOLAR	HU02
3 Gestionar stock	Stock actual y minimo	REPUESTO	HU05
4 Gestionar cotizaciones	Usuario, dolar, cabecera e items	USUARIO, COTIZACION_DOLAR, COTIZACION, DETALLE_COTIZACION, REPUESTO	HU03, HU04

La matriz confirma consistencia de nombres y responsabilidades. La entidad COTIZACION_DOLAR participa en el proceso 4 porque el presupuesto debe conservar el valor usado. USUARIO aporta autoria y control de acceso aunque no sea un almacen comercial destacado en el DFD grafico.

12 Modelo de monetizacion

La alternativa mas adecuada es Software como Servicio con suscripcion mensual por sucursal. Reduce la inversion inicial del cliente y permite mantener el producto actualizado.

Plan	Incluye	Cliente objetivo
Inicial	Hasta 2 usuarios, 1 sucursal, catalogo, dolar, stock y cotizaciones.	Distribuidora pequeña

Plan	Incluye	Cliente objetivo
Profesional	Hasta 10 usuarios, reportes, exportacion e importacion de listas.	PyME en crecimiento
Empresa	Usuarios y sucursales ampliables, API e integraciones.	Red de distribuidores

Los valores monetarios no se fijan en esta etapa porque requieren validar mercado, costos de infraestructura, soporte e impuestos. La prueba de concepto puede ofrecer un periodo de evaluacion y cobrar configuracion o migracion inicial como servicio opcional.

12.1 Indicadores del producto

- Usuarios activos por mes.
- Cotizaciones generadas y confirmadas.
- Tiempo promedio para generar un presupuesto.
- Errores de precio reportados.
- Productos bajo stock minimo.
- Tasa de renovacion de clientes.

13 Diagrama de clases UML

El modelo de clases traslada las entidades del DER a objetos con comportamiento. Los servicios de aplicacion coordinan autenticacion, calculo, persistencia y transacciones.

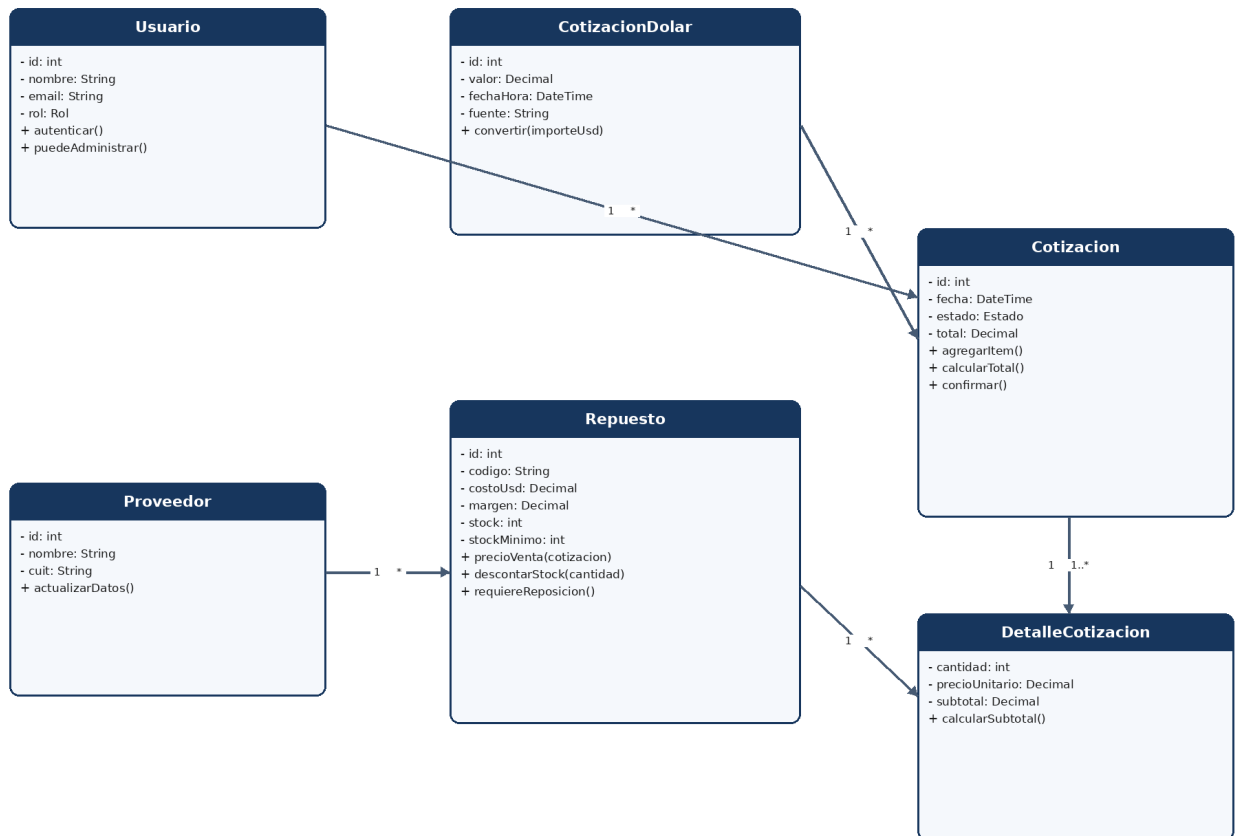


Figura 4. Diagrama de clases UML de StockPar

13.1 Responsabilidades

Clase	Responsabilidad principal
Usuario	Representar identidad, rol y permisos.
Proveedor	Mantener datos del proveedor.
Repuesto	Calcular precio, informar reposicion y modificar stock.
CotizacionDolar	Convertir importes USD a ARS.
Cotizacion	Agrupar detalles, calcular total y confirmar.
DetalleCotizacion	Conservar cantidad, precio unitario y subtotal.

14 Diseño de API REST

La API usa JSON sobre HTTPS, autenticacion mediante token y prefijo /api/v1. Los permisos se validan por rol. Los errores devuelven un codigo HTTP y un cuerpo uniforme con code, message y details.

Metodo	Ruta	Rol	Uso
POST	/auth/login	Publico	Iniciar sesion
GET	/repuestos	Ambos	Buscar y listar
POST	/repuestos	Administrador	Crear repuesto
PUT	/repuestos/{id}	Administrador	Actualizar repuesto
GET	/proveedores	Administrador	Listar proveedores
POST	/proveedores	Administrador	Crear proveedor
GET	/cotizaciones-dolar/ vigente	Ambos	Consultar valor vigente
POST	/cotizaciones-dolar	Administrador	Registrar nuevo valor
GET	/stock/bajo-minimo	Administrador	Listar alertas
POST	/cotizaciones	Vendedor	Crear presupuesto
GET	/cotizaciones/{id}	Ambos	Consultar presupuesto
POST	/cotizaciones/{id}/ confirmar	Vendedor	Confirmar y descontar stock

14.1 Ejemplo de solicitud

POST /api/v1/cotizaciones

```
{ "cliente_nombre": "Taller Norte", "cliente_contacto": "11-5555-0101", "items":  
[{"id_repuesto": 12, "cantidad": 2 }] }
```

14.2 Ejemplo de respuesta

```
{ "id": 104, "estado": "BORRADOR", "cotizacion_dolar": 1450.00, "total": 87000.00, "items":  
[{"codigo": "RP-012", "cantidad": 2, "precio_unitario": 43500.00, "subtotal": 87000.00 }] }
```

14.3 Codigos principales

Codigo	Significado
200	Consulta correcta
201	Recurso creado
400	Solicitud invalida
401	No autenticado
403	Sin permiso
404	Recurso inexistente
409	Conflicto de stock o duplicado
422	Error de validacion
500	Error interno

15 Prototipo de codigo orientado a objetos

El siguiente prototipo en Java muestra la logica central sin depender de una base de datos o framework.

```

public final class Repuesto {
    private final String codigo;
    private final BigDecimal costoUsd;
    private final BigDecimal margenPorcentaje;
    private int stockActual;
    private final int stockMinimo;

    public BigDecimal precioVenta(BigDecimal valorDolar) {
        BigDecimal base = costoUsd.multiply(valorDolar);
        BigDecimal factor = BigDecimal.ONE.add(
            margenPorcentaje.divide(new BigDecimal("100"))
        );
        return base.multiply(factor).setScale(2, RoundingMode.HALF_UP);
    }

    public void descontarStock(int cantidad) {
        if (cantidad <= 0 || cantidad > stockActual)
            throw new IllegalArgumentException("Stock insuficiente");
        stockActual -= cantidad;
    }

    public boolean requiereReposicion() {
        return stockActual <= stockMinimo;
    }
}

```

15.1 Servicio de cotizaciones

```

public Cotizacion crear(Solicitud solicitud, Usuario vendedor) {
    CotizacionDolar dolar = dolarRepo.obtenerVigente();
    Cotizacion cotizacion = new Cotizacion(vendedor, dolar, solicitud.cliente());
    for (ItemSolicitud item : solicitud.items()) {
        Repuesto repuesto = repuestoRepo.buscar(item.idRepuesto());
        BigDecimal precio = repuesto.precioVenta(dolar.valor());
        cotizacion.agregar(repuesto, item.cantidad(), precio);
    }
    cotizacion.calcularTotal();
    return cotizacionRepo.guardar(cotizacion);
}

```

Al confirmar una venta, la aplicacion debe iniciar una transaccion, bloquear o validar las filas de stock, descontar cantidades y cambiar el estado. Si una operacion falla, se revierte todo para evitar inconsistencias.

16 Repositorio GitHub

El repositorio publico recomendado se denomina stockpar. Debe incluir README, documentacion, diagramas, codigo fuente, pruebas y un historial de commits comprensible.

Ruta	Contenido
README.md	Problema, alcance, instalacion, uso y autores.
docs/	Documento, diagramas y decisiones.
src/	Codigo fuente organizado por capas.

Ruta	Contenido
tests/	Pruebas unitarias y de integracion.
api/	Contrato OpenAPI o coleccion de pruebas.
.gitignore	Archivos generados, credenciales y configuracion local.

16.1 Estrategia de commits

- docs: agrega problema y propuesta de valor
- docs: incorpora DFD y DER
- feat: implementa calculo de precio
- feat: agrega generacion de cotizaciones
- test: valida stock insuficiente
- docs: documenta API REST

El enlace real y las capturas del historial deben agregarse cuando el repositorio se publique. No se deben subir contraseñas, tokens, archivos .env ni datos reales de clientes.

17 Pruebas y criterios de calidad

Caso	Precondicion	Accion	Resultado esperado
CP01 Precio	Costo USD 20, dolar 1400, margen 25%	Calcular precio	ARS 35000.00
CP02 Busqueda	Existe RP-012	Buscar por codigo	Muestra datos, precio y stock
CP03 Stock bajo	Actual 3, minimo 5	Consultar alertas	Incluye el repuesto
CP04 Cotizacion	Items validos	Crear cotizacion	Guarda cabecera, detalles, dolar y total
CP05 Stock insuficiente	Stock 1, pedido 2	Confirmar venta	Rechaza sin descontar stock
CP06 Seguridad	Usuario Vendedor	Crear repuesto	Responde 403
CP07 Historico	Se actualiza el dolar	Abrir cotizacion anterior	Mantiene valor y precios originales

17.1 Requisitos no funcionales

- Seguridad: HTTPS, hash robusto de contraseñas, roles y auditoria.
- Rendimiento: busquedas habituales en menos de dos segundos bajo carga normal.
- Disponibilidad: respaldo diario y procedimiento de recuperacion.
- Usabilidad: formularios claros, validaciones cercanas al campo y diseño adaptable.
- Mantenibilidad: capas separadas, pruebas automatizadas y documentacion.
- Integridad: transacciones para confirmar ventas y tipos DECIMAL para dinero.

18 Conclusiones

El diseño de StockPar responde al problema identificado y mantiene coherencia entre necesidades, procesos, datos y objetos. La incorporacion de la cotizacion del dolar como dato historico de cada presupuesto es la decision central del modelo: permite recalculer precios actuales sin alterar operaciones anteriores.

El MVP puede implementarse de manera gradual. Primero debe asegurar catalogo, dolar, stock y cotizaciones; luego puede incorporar importacion de listas, reportes, facturacion e integraciones. La trazabilidad y los casos de prueba ofrecen una base verificable para continuar con el desarrollo.

19 Bibliografia y fuentes consultadas

- Boxer Gestion. Sitio oficial. <https://boxergestion.com> (consulta: septiembre de 2026).
- Tezzi Comercial. Sitio oficial. <https://tezzi-comercial.com.ar> (consulta: septiembre de 2026).
- Object Management Group. UML 2.5.1 Specification. <https://www.omg.org/spec/UML/2.5.1>
- OpenAPI Initiative. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>
- Fielding, R. Architectural Styles and the Design of Network-based Software Architectures. 2000.

20 Anexos y evidencias a incorporar

Evidencia	Estado del contenido	Accion final
Captura del tablero Trello	Diseño definido	Crear tablero y pegar captura.
Enlace al repositorio GitHub	Estructura definida	Publicar y agregar URL real.
Capturas de commits	Plan definido	Realizar commits y anexar evidencia.
Ejecucion del prototipo	Codigo base incluido	Compilar y capturar pruebas.
Presentacion oral	Contenido disponible	Preparar diapositivas si la catedra las solicita.

Estas evidencias dependen de cuentas externas y deben reflejar acciones reales. El documento no inventa enlaces ni capturas.